MODEL ANSWERS - Generative AI & ML Fundamentals - Mid-Term (Sample)

Answer 1:
Overfitting is when a model is too simple to learn the patterns in the data, so it does badly on both training and test data. It is reduced by training for more epochs and adding more layers.

Answer 2:
Each input token embedding x is multiplied by learned matrices W_Q, W_K, W_V to give a query q, key k and value v. The score between token i and token j is q_i.k_j, computed for all pairs as QK^T. Scores are divided by sqrt(d_k) and passed through a softmax over j, giving weights that are positive and sum to 1. The output for token i is the weighted sum of all value vectors: Attention(Q,K,V)=softmax(QK^T/sqrt(d_k))V. Scaling is needed because the dot product of two d_k-dimensional vectors has variance proportional to d_k; large scores make the softmax very peaked (saturated), so gradients become extremely small and training becomes unstable.

Answer 3:
Precision = 40/(40+10) = 0.80. Recall = 40/(40+10) = 0.80. F1 = 2x0.8x0.8/(0.8+0.8) = 0.80. Accuracy = (40+30)/100 = 0.70.

Answer 4:
(a) A GAN trains a generator G and a discriminator D in a minimax game: D learns to tell real from fake, G learns to produce samples that fool D. A diffusion model defines a forward process that adds Gaussian noise over many steps and trains a network to predict the noise (i.e. denoise) at each step. (b) GAN training is notoriously unstable - the two networks must stay balanced and training can diverge or oscillate. Diffusion training is stable because it minimises a simple mean-squared error. (c) GANs often suffer from mode collapse, producing limited variety. Diffusion models cover the distribution better and give more diverse samples. (d) GANs produce a sample in one forward pass, so they are fast; diffusion models need tens to thousands of denoising steps, so sampling is slower (mitigated by DDIM, distillation or consistency models).

Answer 5:
Offline, documents are split into chunks, each chunk is converted to an embedding vector and stored in a vector database. At query time the user question is embedded, the top-k most similar chunks are retrieved, and they are inserted into the prompt so the LLM answers using that context. Failure 1: retrieval misses the relevant passage (poor chunking, vocabulary mismatch) - mitigate with hybrid BM25 + dense search and a cross-encoder re-ranker. Failure 2: the LLM hallucinates or ignores the retrieved context - mitigate by instructing it to answer only from context and cite chunks, abstain when evidence is missing, and run a faithfulness check on the output.